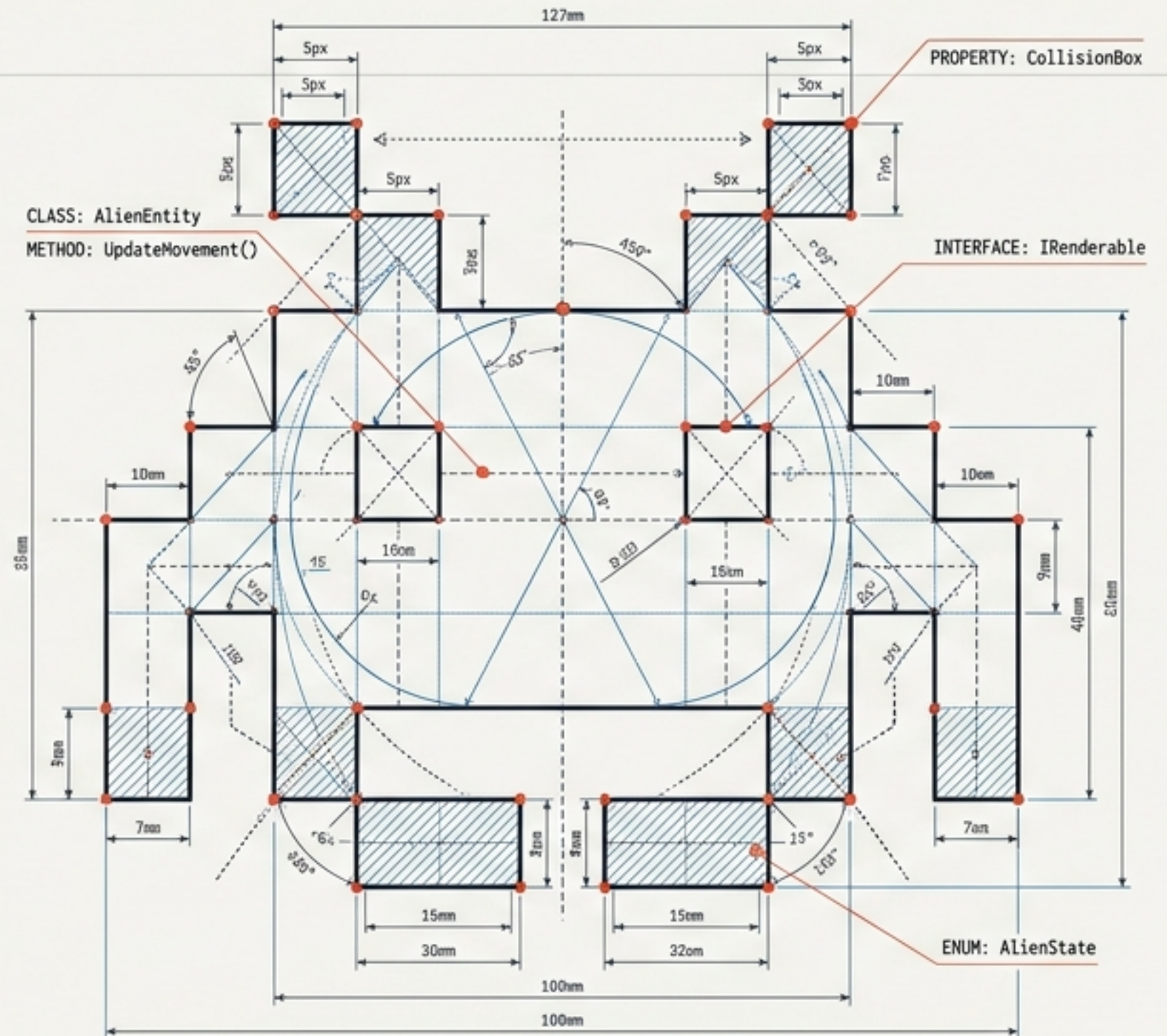


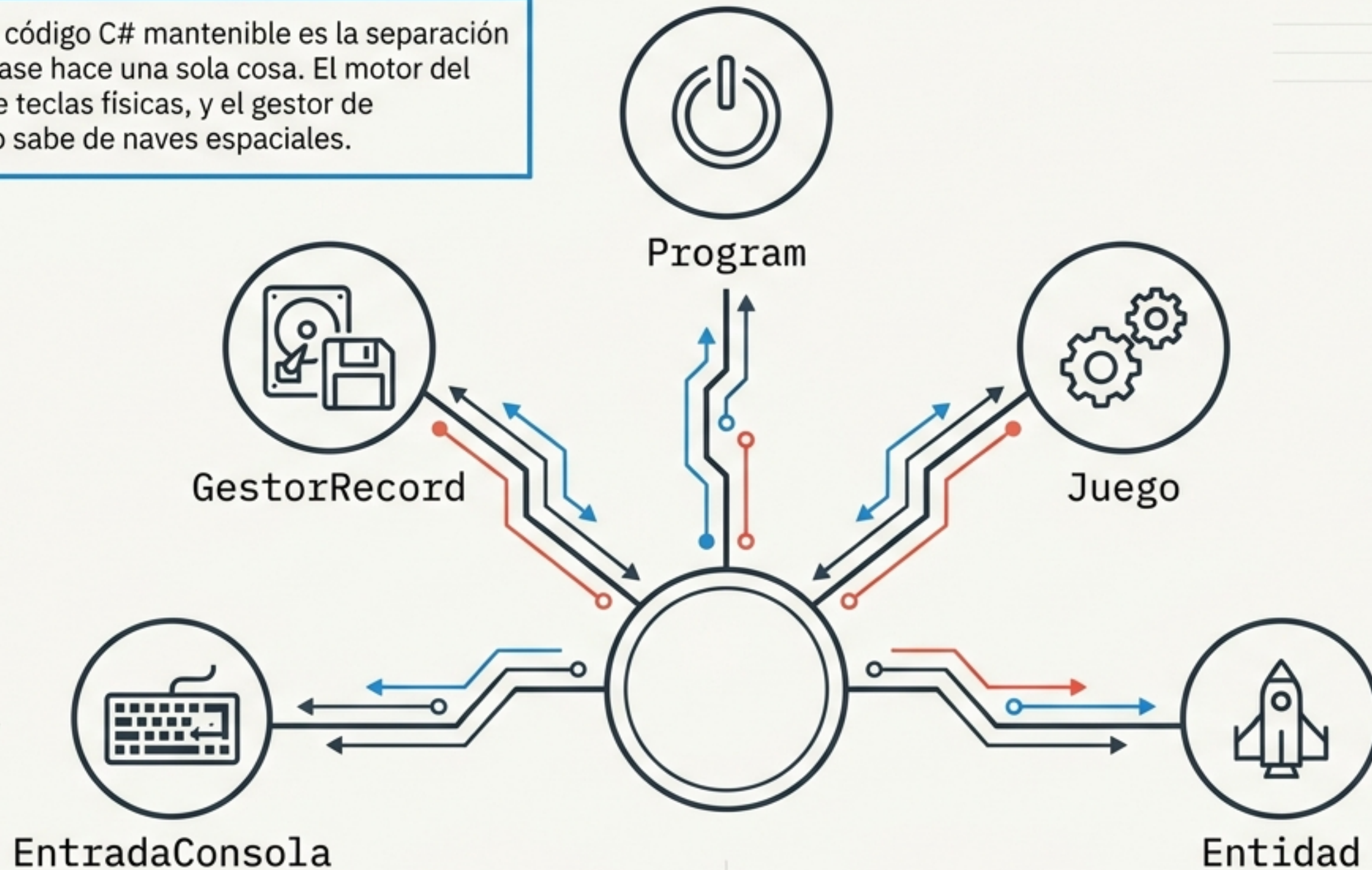
Arquitectura de Space Invaders en C#

Deconstruyendo un motor de juego infinito a través de patrones de diseño limpios y Programación Orientada a Objetos.



El principio de responsabilidad única

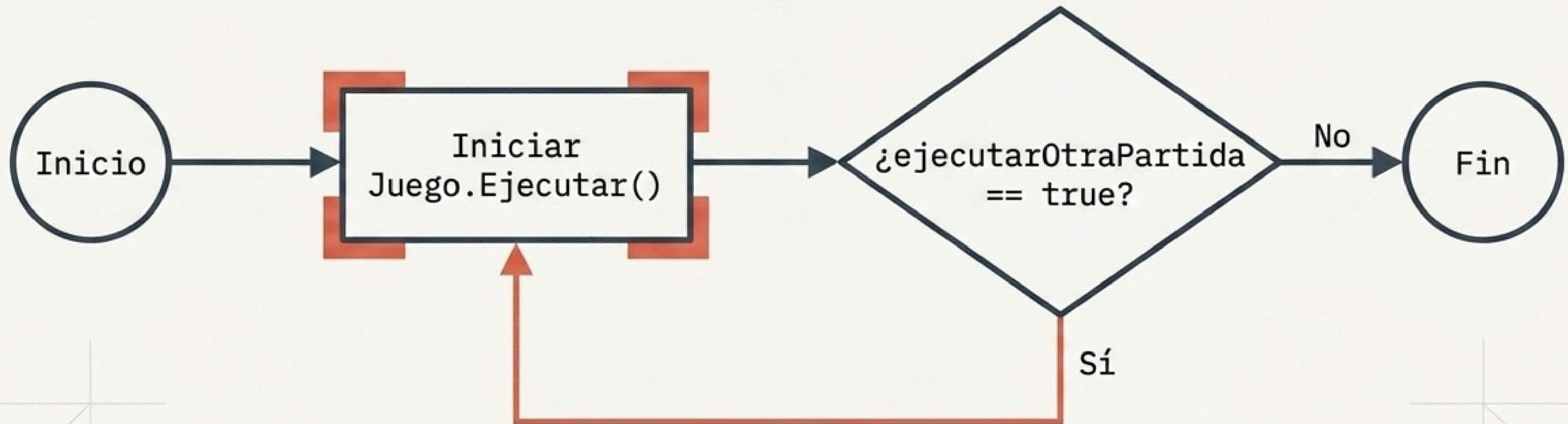
El secreto de un código C# mantenible es la separación estricta. Cada clase hace una sola cosa. El motor del juego no sabe de teclas físicas, y el gestor de puntuaciones no sabe de naves espaciales.



El controlador maestro

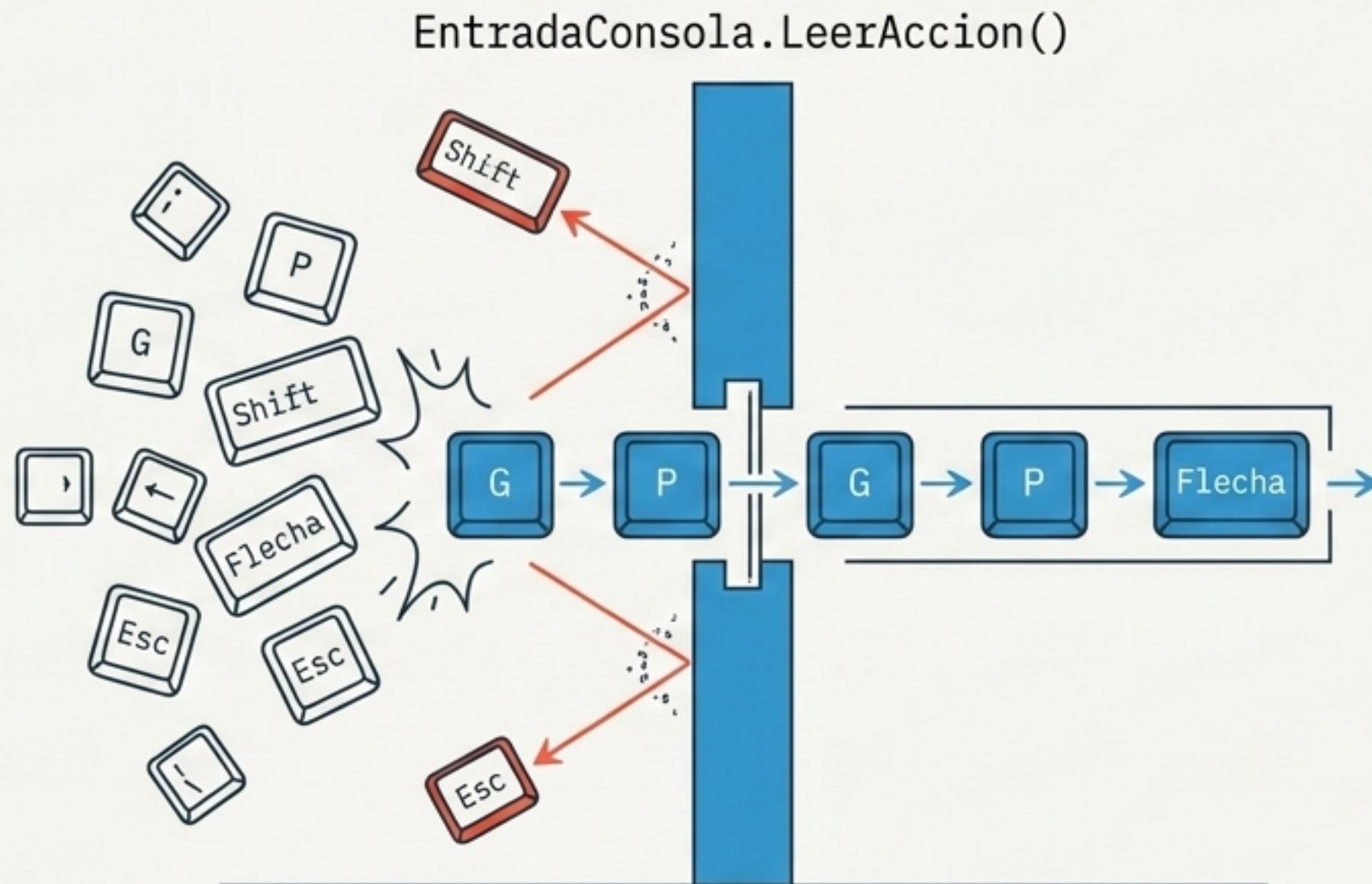
La clase Program es diminuta por diseño. Su única responsabilidad es el ciclo vital básico de la aplicación.

Al extraer todas las reglas concretas hacia Juego, el punto de entrada estático solo se encarga de iniciar partidas y preguntar si el usuario desea jugar de nuevo.



Aislando el hardware del juego

Si el motor del juego lee teclas directamente, la lógica se contamina con validaciones de errores.



Clase Estática

EntradaConsola no almacena estado, se invoca directamente.

Bloqueo Seguro

Utiliza un bucle while que atrapa la ejecución hasta recibir un input válido. Las teclas incorrectas rebotan y no consumen turnos de forma accidental.

Trabajando con intenciones, no con hardware

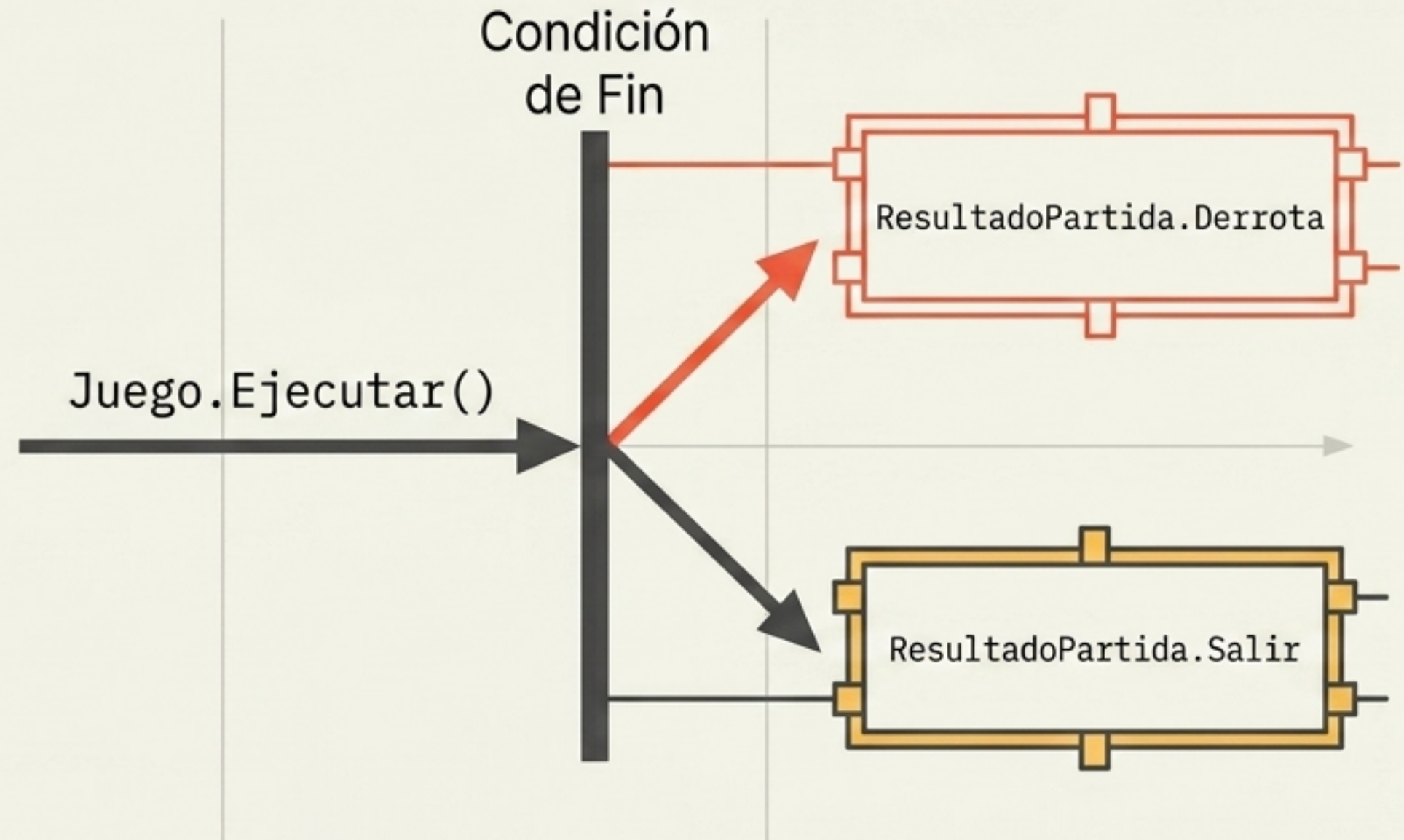
El juego no sabe qué es una "barra espaciadora". Gracias a la enumeración `AccionJugador`, si mañana adaptamos el juego a un mando de consola, la lógica de movimiento en el motor no cambiará ni una sola línea.



Resultados determinísticos

En esta arquitectura, `Juego.Ejecutar()` no tiene un estado de "Victoria" porque no hay un nivel final.

La partida finaliza y devuelve el enum `ResultadoPartida` solo si un alienígena invade la base (**Derrota**) o el jugador aborta. Program usa esta señal limpia para decidir la siguiente pantalla.

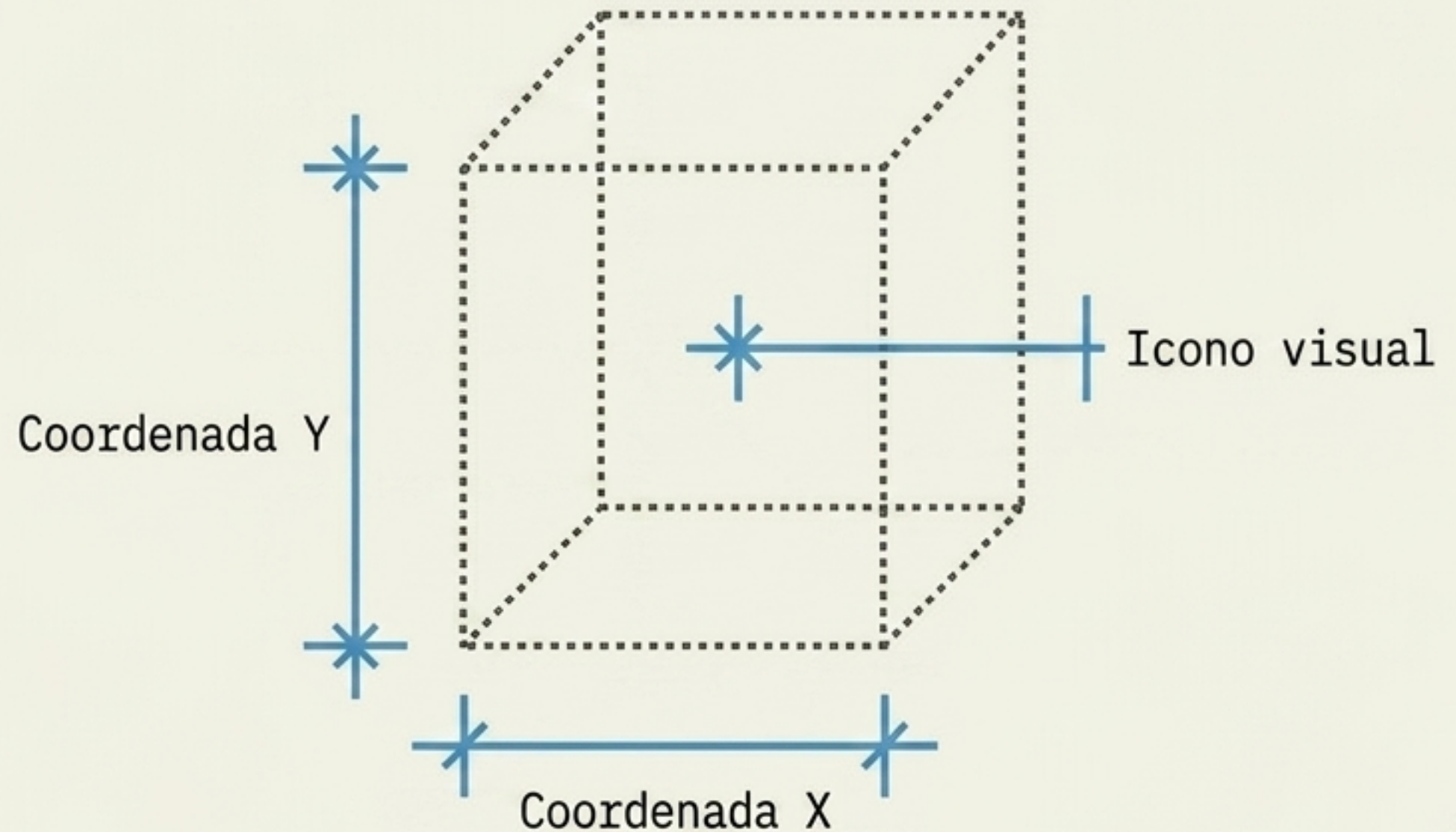


El paradigma de la herencia

Identificar lo común. Naves, alienígenas y balas necesitan saber dónde están y cómo se ven.

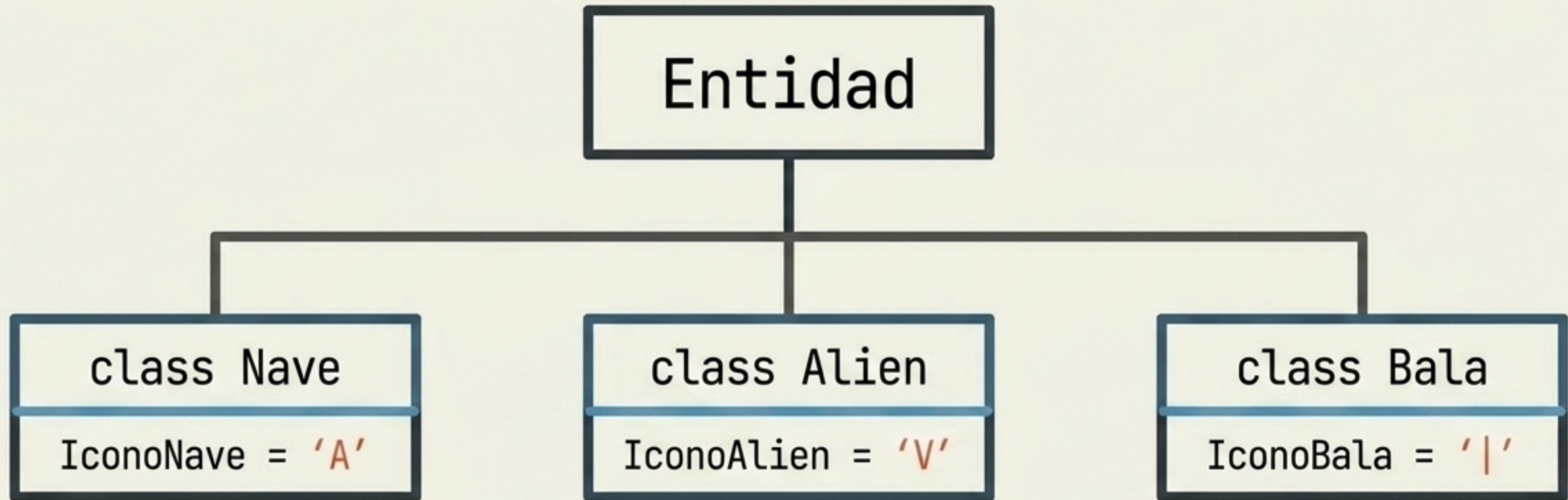
```
abstract class Entidad
```

La clase base es abstract. Su constructor es protected: nunca instanciamos una "Entidad" genérica, solo sus especializaciones. Esto evita repetir código.



Especializando a los actores

DRY (Don't Repeat Yourself). Las clases derivadas heredan las coordenadas automáticamente, pero definen su propia identidad visual inmutable mediante `private const char`.



Matriz de Entidades

La magia del polimorfismo. Al heredar de Entidad, el motor del juego puede almacenar naves, aliens y balas en las mismas listas y actualizarlos de forma unificada.

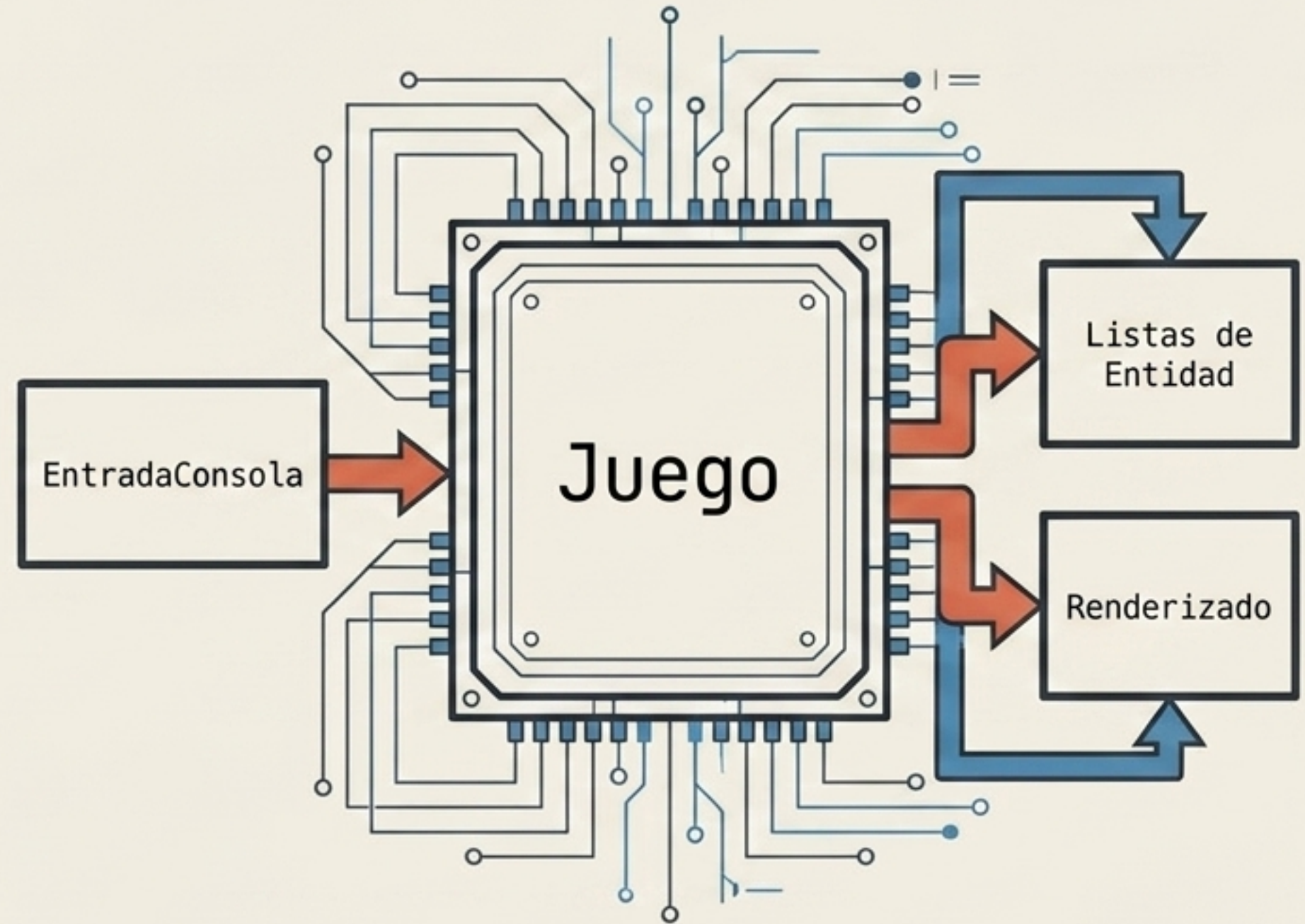
Clase	Herencia	Constante Visual	Comportamiento
Nave	Entidad ✓	'A'	Movimiento lateral
Alien	Entidad ✓	'V'	Descenso autónomo
Bala	Entidad ✓	' '	Ascenso lineal

El motor principal

La clase Juego es el orquestador absoluto. Es el único lugar que conoce el estado completo del tablero en un instante dado.

Responsabilidades Clave

1. Crea oleadas.
2. Procesa AccionJugador.
3. Detecta colisiones.
4. Manda a dibujar el tablero.



Ingeniería de la dificultad infinita

¿Cómo escalar sin programar niveles nuevos?
Alterando variables en tiempo de ejecución.

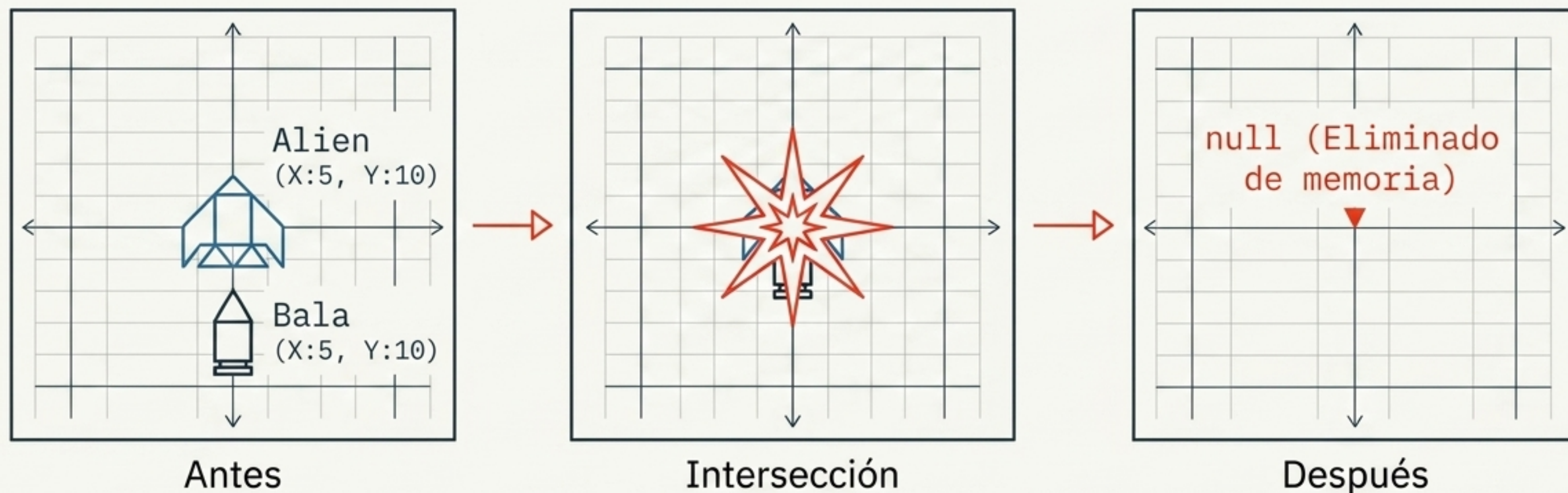
Volumen: Cada nivel suma un alienígena (hasta llenar 12 columnas).

Frecuencia: Los enemigos se mueven con mayor rapidez.

Salto: En niveles extremos, avanzan múltiples filas de golpe.

Detección de impactos y ciclo vital

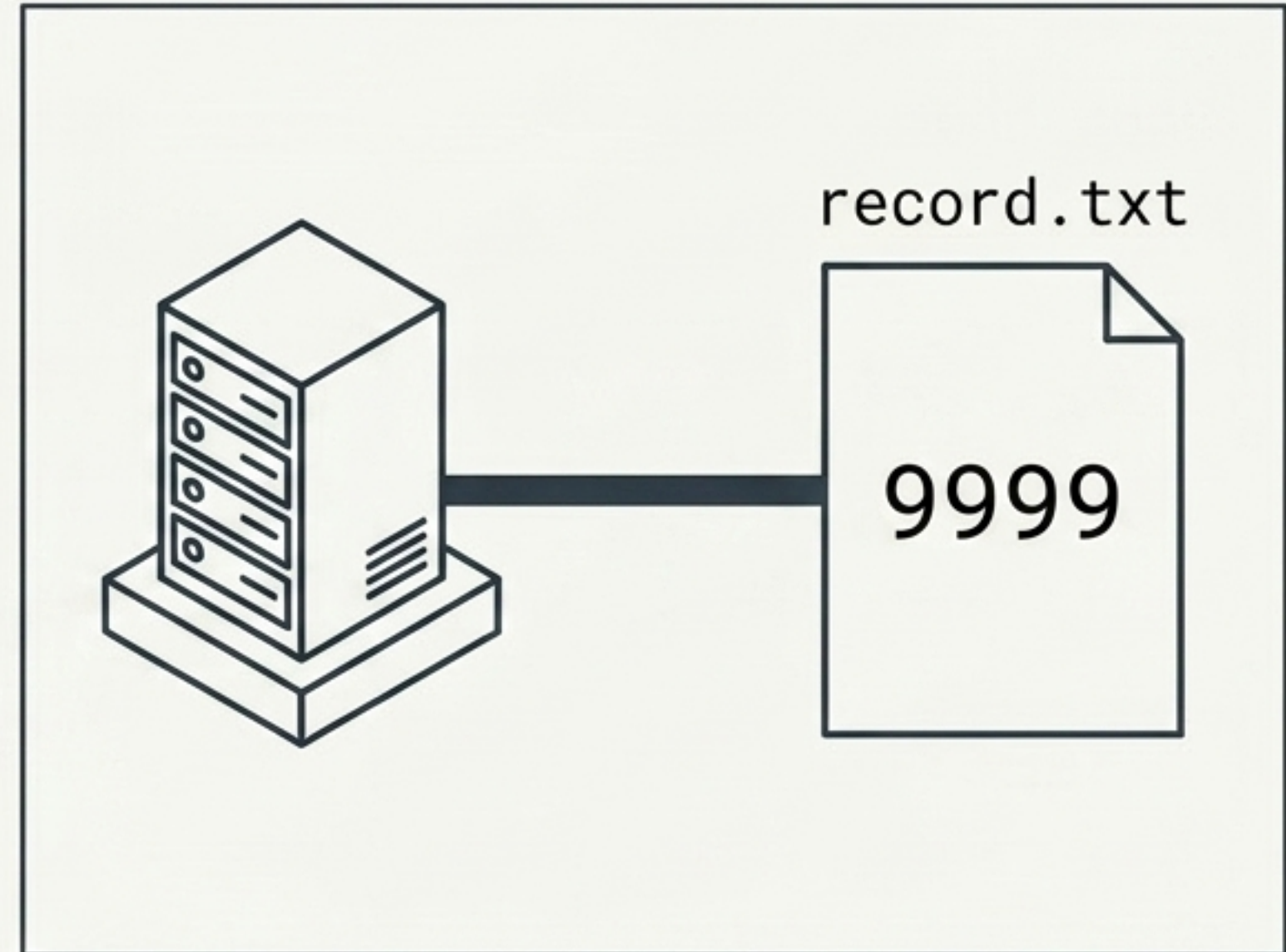
La lógica de interacción vive en Juego. Si las coordenadas X e Y de una bala coinciden con las de un alienígena, ambas entidades se eliminan de la lista de actualización. Si la lista de alienígenas queda vacía, se genera la siguiente oleada.



Persistencia segura

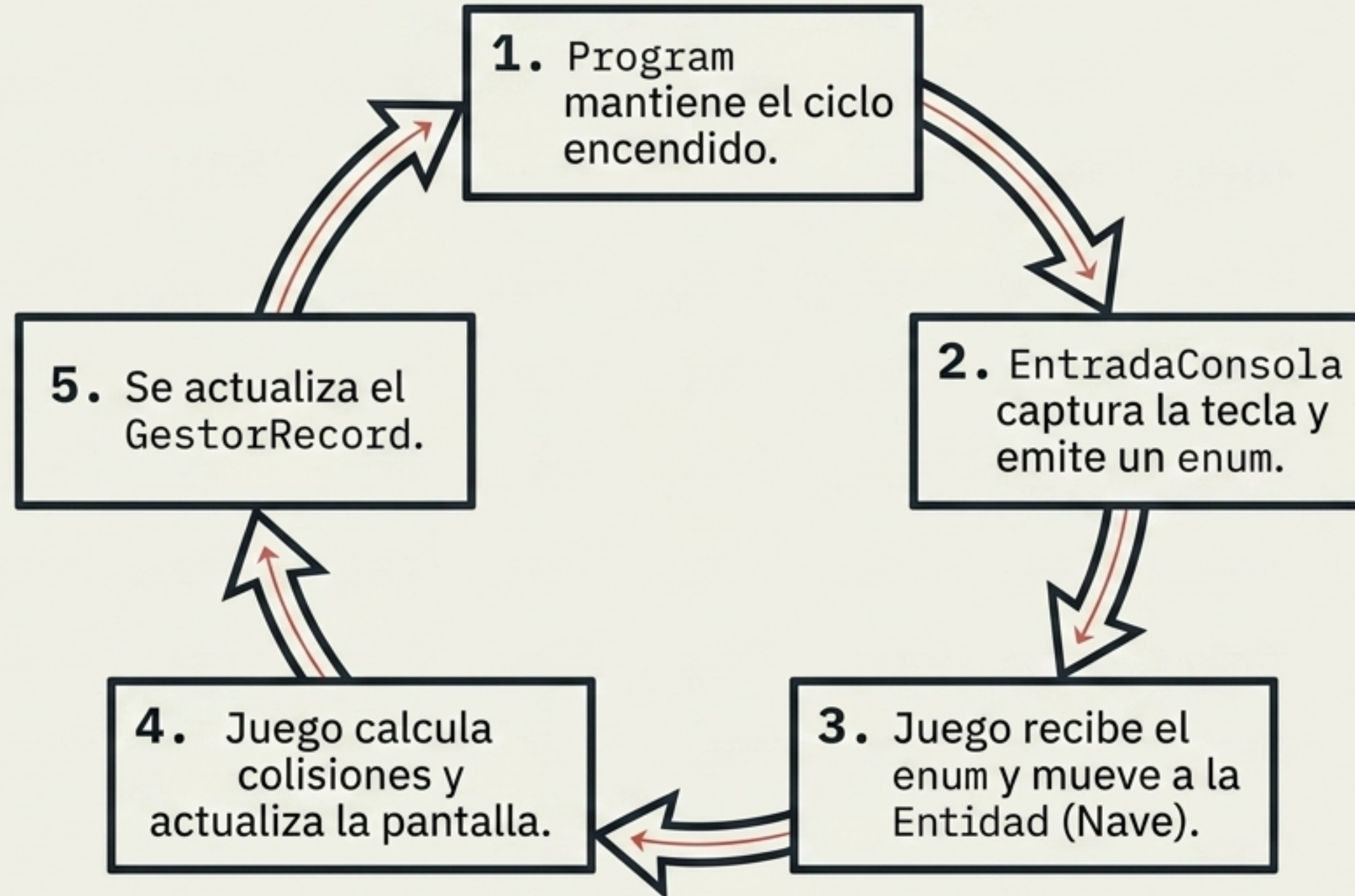
La clase static `GestorRecord` se responsabiliza exclusivamente de leer y escribir la puntuación más alta en disco.

Al externalizar esta tarea, el motor de juego (Juego) no se ensucia con lógica de ficheros, permisos de sistema, o manejo de excepciones de I/O.



Síntesis arquitectónica

Todas las piezas del motor trabajando juntas. Completamente
Completamente desacopladas, altamente mantenibles.



Conclusiones para el desarrollador C#

1

Regla 1 (Abstracciones protegen tu código)

Usa enums para traducir entradas físicas a intenciones lógicas abstractas.

2

Regla 2 (DRY con Herencia)

Si varias clases comparten estado (posiciones, sprites), consólidalas en una `abstract class`.

3

Regla 3 (Sistemas sobre Datos Escritos)

Reglas matemáticas simples en una clase orquestadora escalan infinitamente mejor que diseñar niveles a mano de forma rígida.